Mizan.ai

RAG Embeddings, Chunking & Pipeline Architecture

The end-to-end technical design for Feature A's knowledge base: how source documents are extracted, chunked, embedded, and stored; how tables and cross-references are handled; the split between the offline (local) ingestion pipeline and the online (Modal) inference pipeline; and the retrieval and fallback logic that governs every answer. Companion to the RAG Data Source Strategy document.

July 2026 | Internal reference document | Build planning

Embedding model	BGE-M3, 1024-dim, dense vectors only
Vector store	Qdrant (single collection, payload-filtered)
Structured store	SQLite (table rows + registry)
Chunking	Structural (article/clause-level), no overlap, in-language context headers
Table handling	Descriptions embedded in Qdrant, actual rows in SQLite (table-pointer pattern)
Cross-references	One-hop, composite-key resolution, article/table type-branched
Two pipelines	Offline ingestion (local, no Modal) + online inference (Modal)
Fallbacks	Wrong jurisdiction (pre-search) + nothing-found (post-search)

1. Two-Pipeline Architecture

The system is split into two independent pipelines with different runtimes, triggers, and responsibilities. This decouples slow, infrequent knowledge-base construction from fast, always-on user-facing inference.

	Offline / Ingestion	Online / Inference
Runs on	Local machine (8GB VRAM laptop)	Modal (serverless)
Trigger	Manual (monthly, or when ZATCA amends a document)	Every user query
Embedding	Local BGE-M3 (same checkpoint/config as Modal)	Modal BGE-M3 /embed endpoint
External calls	Claude API only (table descriptions + reference review)	None beyond Modal-hosted models
Writes	Qdrant collection + SQLite store	Nothing — read-only consumer
Answer model	N/A	QwenLite (/generate-lite)

Critical consistency requirement: the offline and online pipelines embed into the same vector space, so both must use an identical BGE-M3 checkpoint and config (`use_fp16=True`). If they drift, retrieval quality degrades silently — no error, just worse matches. This is verified manually and belongs on the ingestion checklist.

2. Offline / Ingestion Pipeline

Runs locally, on demand. Turns source documents into embedded chunks (Qdrant) and structured table data (SQLite). Makes no calls to Modal; the only external dependency is the Claude API, used once per table/document during ingestion (never at inference time).

#	Step	What happens
1	**Fetch**	Pull source files from the cataloged ZATCA/SOCPA URLs into a local raw/ folder, tagged with fetch date.
2	**Type-detect & route**	PDF → Docling; XLSX/CSV → direct pandas parse; XSD → copied to the Feature E codebase, skips the KB pipeline entirely.
3	**Extract (Docling)**	Convert PDFs to structured markdown/JSON, preserving table structure. Language (AR/EN) is known from the source path, not re-detected.
4	**Quality gate**	Flag tables with merged cells or dense numeric data for full review. On error: correct the extracted data in place (NOT re-run through Docling — its merged-cell failure is systematic and would repeat). Track corrected tables so review isn't redone every month.
5	**Chunk (prose)**	Split each legal document into one chunk per article (regex on “Article N” / المادة). Sub-split only if an article exceeds ~500 words. Embed an in-language context header into each chunk's text. No overlap.
6	**Tables → SQLite**	Store rows in table_rows; register each table in tables_registry with full metadata. Decide granularity by row count: small tables (~<20 rows) = one whole-table pointer; large tables (Data Dictionary, code lists) = per-row/group pointers.
7	**Table descriptions (Claude)**	One short natural-language description per table (or per row-group for large tables), authored once. This is the text that gets embedded as the table-pointer.
8	**Detect cross-references**	Regex first pass (“Article N”, both languages) + Claude review pass for indirect/spelled-out references. Classify each as type:article or type:table. Attach as references metadata on the citing chunk.
9	**Embed (local BGE-M3)**	Embed each prose chunk's text and each table description. Same checkpoint/config as Modal.

#	Step	What happens
10	**Write to Qdrant**	Upsert prose chunks (content_kind:prose) and table-pointer chunks (content_kind:table_pointer) with full payload.

Pipeline-wide ordering constraint: all tables must be registered (steps 6–7) across every document before cross-reference detection (step 8) runs, so that a reference to a table — possibly defined in a different document — can resolve to the correct table_ref. Table registration is a pipeline-wide phase, not a per-document step.

3. Online / Inference Pipeline

Runs on Modal, on every user query. Read-only against Qdrant + SQLite. Makes one honest judgment — "did the search find anything useful?" — rather than trying to classify scope before searching, which can't be done reliably.

#	Step	What happens
1	**Receive query**	User message arrives via Feature A's chat flow.
2	**Detect language**	Arabic or English (reuses existing detect_language()).
3	**Non-KSA country check**	Cheap keyword check for an explicit non-KSA GCC country (Bahrain, UAE, Oman, Qatar, Kuwait + Arabic names). If matched → wrong-jurisdiction fallback, no search needed. This is the ONLY pre-search scope check — reliable because it's keyword-based, not semantic.
4	**Embed query**	Call the Modal BGE-M3 /embed endpoint.
5	**Search Qdrant**	Same-language-first: filter language = query_language. If results are empty/thin, retry once without the language filter (cross-language fallback).
6	**Nothing found?**	If no sufficiently relevant results come back even after the retry → the single "we didn't find an answer" fallback. Covers both out-of-scope and insufficient-detail cases — both look identical here (nothing relevant retrieved), so one honest message handles both.
7	**Resolve table pointers**	For any retrieved content_kind:table_pointer chunk, look up its table_ref in SQLite and fetch the relevant row(s). A whole small table's rows count as ONE budget item, not per-row.
8	**Resolve cross-references**	For each retrieved chunk's references (one hop only): type:article → Qdrant lookup by composite key (document_type + inherited version_label + article_number); type:table → SQLite lookup by table_ref.
9	**Apply context cap**	Cap total chunks (original + resolved) at 6, prioritizing original retrieval matches over their resolved references if the cap is hit.
10	**Cross-language wrapper**	If the answer relies on fallback-language (English) content for an Arabic query, wrap it in a same-language explanatory message. Do NOT auto-translate the content; the user may trigger translation themselves.
11	**Generate answer (QwenLite)**	System prompt enforces: default to current version + explicit labeling of any non-current content; mandatory 4-part citation on every claim; resolve table pointers before answering; no silent translation.
12	**Return response**	Answer + citations, or a fallback message.

Conversation history: reuses the existing chatbot's in-memory logic — only (user query + final response) pairs are kept, capped at 8 turns, consistent with the deployed langgraph_chatbot.py dual-window approach.

4. Chunking Strategy

DECISION: Structural chunking (article/clause-level), NOT fixed-token windows. Legal meaning lives at the article level; arbitrary splits risk fragments that read as true but are incomplete.

Chunk unit per document type

Document type	Chunk unit
VAT Law, Implementing Regs, Zakat Regs, GCC Agreement	One article per chunk (sub-split if >~500 words)
SOCPA standards	One standard/section per chunk; worked examples kept attached to their rule
ZATCA sector guidelines	One guidance point/example per chunk
E-Invoicing XML rule tables (BR-S, BR-Z, ...)	One rule (or tight rule-group) per chunk, via table-pointer pattern
Penalty / rate-exemption matrices	Handled as tables (SQLite + pointer), not prose chunking

Two cross-cutting rules

- **In-language context header, embedded in the chunk text** — e.g. “VAT Implementing Regulations, Article 47:” prefixed to the article body (in the same language as the body, so the language field stays clean). The header is embedded into the vector, so retrieval matches on chapter/topic context, not just bare clause wording.
- **No overlap** between structurally-chunked legal articles. Overlap exists in standard RAG to compensate for arbitrary token splits; chunking on natural article boundaries makes it unnecessary, and it would only create near-duplicate chunks in Qdrant.

5. Table Handling — The Table-Pointer Pattern

A table's meaning lives in the relationship between cells — “SAR 50,000” only means something because of the row it sits in. Flattening a table into embedded prose risks attaching the wrong penalty to the wrong violation while every extracted word is individually correct. The table-pointer pattern avoids this.

How it works

- The actual table data stays in SQLite — exact, complete, unmodified.
- A short natural-language **description** of the table is embedded into Qdrant (not the table itself), tagged content_kind:table_pointer with a table_ref.
- At query time, semantic search matches the description (on meaning, not exact table name); the system then follows table_ref to fetch the real rows from SQLite.
- The embedding's only job is “recognize what this table is about” — a task embeddings are good at — while the factual retrieval becomes a precise lookup, not a similarity guess.

Granularity by table size

- **Pattern A — small tables** (penalty matrix ~15 rows, rate/exemption matrix): one pointer for the whole table; resolution fetches all rows (still small; counts as one budget item).
- **Pattern B — large tables** (Data Dictionary 100+ fields, code lists): sharded into per-row/group pointers, each with its own description; resolution fetches only the matched pointer's rows, never the whole table. Decided by a row-count threshold (~20) at ingestion.

6. Chunk Metadata Schema

Every chunk — prose or table-pointer — carries the shared citation backbone. Table-pointers add two fields (content_kind, table_ref); prose chunks carry article_number instead.

Field	Applies to	Purpose
`content_kind`	all	“prose” | “table_pointer” — tells the answer step whether to answer directly or resolve a table
`document_type`	all	e.g. “VAT Implementing Regulations” — part of the cross-reference composite key
`article_number`	prose	The article's own identity — the resolution target for cross-references
`table_ref`	table_pointer	SQLite key to fetch the actual rows
`jurisdiction`	all	“KSA” (v1 is KSA-only)
`language`	all	“ar” | “en” — drives same-language-first retrieval filtering
`version_label`	all	e.g. “current” | “amended 15 June 2023”
`is_current`	all	Boolean — retrieval defaults to current
`effective_start_date, effective_end_date`	all	Date range the version was/is in effect (end_date null = still current)
`references`	prose	List of {type:article|table, ...} — the cross-reference targets
`source_site / source_url`	all	Citation provenance
`text`	all	The embedded text (article body for prose; description for table-pointer)

7. Cross-Reference Resolution

Legal text constantly cites other articles (“subject to Article 25”) and tables (“penalties per the Schedule”). A chunk retrieved with a bare reference to Article 25, without Article 25 present, gives the model an incomplete picture. Resolution fixes this.

The composite-key problem

“Article 25” is not unique — it exists in the VAT Regs, the Zakat Regs, and the GCC Agreement, each meaning something different. Resolution keys on `document_type + version_label + article_number`, defaulting to the SAME document and version as the citing chunk (the version_label is inherited at resolution time, not stored in the reference). Cross-document references are resolved only when the text explicitly names the other law.

Type-branched lookup

Each reference is tagged type:article or type:table during the Claude review pass. At resolution:

- **type == “table”** → SQLite lookup by table_ref
- **type == “article”** → Qdrant metadata lookup by composite key (document_type + inherited version_label + article_number)

One hop only: if Article 25 itself references Article 10, that second hop is NOT followed — one level of resolution catches the vast majority of “incomplete without its direct dependency” cases without context bloat or reference-loop risk. Resolved content counts toward the 6-chunk cap.

8. Retrieval Logic & Fallbacks

Same-language-first retrieval

- Arabic query → search Arabic chunks only (filter language=ar). English query → English chunks. Arabic is the legally authoritative text.
- If same-language retrieval is empty/thin → retry once without the language filter (cross-language fallback), so content that exists only in the other language can still surface.
- On cross-language fallback, the retrieved content is shown UNTRANSLATED, wrapped in a same-language message explaining it wasn't available in the query language. Mizan.ai never auto-translates legal content — the user may trigger translation themselves.

Fail-closed principle

Retrieval and generation fail closed, never guess. There are two fallback conditions:

Fallback	Trigger	When decided
Wrong jurisdiction	Explicit non-KSA GCC country in the query	Pre-search (keyword check)
Nothing found	Search returns nothing sufficiently relevant (covers both out-of-scope and insufficient-detail)	Post-search (relevance judgment)

Earlier the design had a separate pre-search “out-of-scope” classifier. It was removed: detecting out-of-scope reliably before searching is not possible (customs and VAT share vocabulary), and the cleanest signal for both out-of-scope and insufficient-detail is the same — nothing relevant came back. One post-search check now handles both.

Fallback messages (bilingual)

Wrong jurisdiction — English:

Mizan.ai currently supports Saudi Arabia (ZATCA/KSA) tax and compliance guidance only. We don't yet cover other GCC countries. If you need support for [detected country], please reach out to our team — we're always looking to understand where to expand next.

Wrong jurisdiction — Arabic:

بعد. إذا كنت بحاجة إلى دعم لـ الدولة المكتشفة، يرجى التواصل مع فريقنا - نحن نتطلع دائما إلى معرفة أين نوسع خدماتنا.
الخاصة بالمملكة العربية السعودية هيئة الزكاة والضريبة والجمارك فقط، ولا يغطي دول مجلس التعاون الخليجي الأخرى
يدعم . حاليا الإرشادات الضريبية والامتثال

Nothing found — English:

I wasn't able to find a clear answer to this in Mizan.ai's current knowledge base. Rather than guess, I'd rather be upfront that this isn't covered with enough detail yet. For help with this specific question, please reach out to our team.

Nothing found — Arabic:

هذا الموضوع غير مغطى بتفصيل كاف حتى الآن. للحصول على المساعدة بخصوص هذا السؤال تحديدا، يرجى التواصل مع فريقنا.
من العثور على إجابة واضحة لهذا السؤال ضمن قاعدة معارف . الحالية. بدلا من التخمين، أفضل أن أكون واضحا بأن
لم أتمكن

9. Design Validation — Issues Found & Resolved

The design was stress-tested before finalizing. The issues found and their resolutions:

Issue	Resolution
Article-number collisions across documents	Composite key: document_type + version_label + article_number. Default to same-document, same-version.
References can point to tables, not just articles	Typed references (type:article | type:table); branch the lookup by type.
Context bloat from pairing + references + one-hop	Hard cap of 6 chunks post-expansion; originals prioritized over resolved references. AR/EN pairing pull dropped (same-language-only search).
Claude review timing / no-Modal-calls constraint	Claude review runs offline only, recurring per amendment. It's a Claude API call, not a Modal call — constraint intact.
Version precision for historical queries	Added effective_start_date / end_date alongside is_current and version_label.
Whole-table fetch blowing the context budget	Small tables = one pointer (all rows, one budget item); large tables = sharded per-row/group pointers, fetch only matched rows.
Pre-search out-of-scope classification unreliable	Removed. Only explicit non-KSA country is checked pre-search; everything else is a post-search “nothing found” judgment.
Multi-turn conversation	Reuse existing in-memory chatbot logic; keep (query + response) pairs only, capped at 8 turns.
Table-registration ordering across documents	All tables registered before cross-reference detection runs, so table references resolve correctly.
Quality-gate failure action	Correct extracted data in place (never re-run Docling on a systematic failure); track corrected tables to avoid monthly rework.

10. Manual Ingestion Checklist (per re-ingestion)

Because versioning and embedder-consistency are handled manually for v1, these steps must be run by hand each time a document is re-ingested after a ZATCA amendment:

- Confirm the local BGE-M3 version/config matches the Modal deployment before embedding.
- For the amended document: mark the previously-current version's chunks is_current = false and set their effective_end_date.
- Add the new version's chunks with is_current = true and the correct effective_start_date.
- Re-run the quality gate on any changed tables; reuse prior corrections for unchanged tables (check the corrected-table tracking).
- Register all tables before running cross-reference detection.
- Spot-check the highest-stakes tables (penalty matrix, rate/exemption matrix) against the source PDF.

Companion to the Mizan.ai RAG Data Source Strategy document. Together these cover the complete Feature A knowledge-base design: what goes in (source strategy) and how it is built, stored, and served (this document). Revisit as ZATCA/SOCPA publish updates.